

Práctica 1: Planificación estática de instrucciones

Arquitecturas de computación avanzadas

Curso 24/25

1. Objetivos de la práctica

- Repasar conceptos básicos del pipeline segmentado de 5 etapas de MIPS.
- Comprender la técnica de planificación estática de desenrollamiento de bucles.
- Comprender la técnica de planificación estática de software pipelining.

2. Desarrollo de la práctica

Durante la primera parte del desarrollo de la práctica el alumno debe repasar conceptos básicos de pipeline segmentado de MIPS, como la anticipación, la resolución de saltos y la predicción de los mismos. En la segunda parte, se estudiarán diversos casos de planificación estática de instrucciones en procesadores segmentados multiciclo, utilizando las técnicas de *desenrollamiento de bucles* y de *software pipelining*. Para ello, se utilizará el simulador *MARS-F*, disponible en el Campus Virtual de la asignatura y que será utilizado tanto como *IDE* de programación de MIPS como de simulador de procesador MIPS con pipeline segmentado. En el Campus Virtual se incluye documentación para la utilización del simulador, así como los ficheros fuente necesarios para realizar la práctica.

Consideraciones extra

- En los ejercicios de reorganización de código, se pueden reorganizar todas las instrucciones, no solamente las implicadas en el desenrollamiento de bucles o el software pipelining.
- Al reorganizar el código, no hace falta limitar el uso de los registros a los registros temporales $\$tX$. Los registros $\$sX$, $\$aX$ y $\$vX$ también pueden utilizarse.
- Para reducir los riesgos estructurales en la etapa *Writeback*, las instrucciones de almacenamiento (*sw*, *sb*, etc.) y las bifurcaciones (*bne*, *bne*) no utilizan esta etapa.

2.1. Entrega de la práctica

La fecha límite de entrega sin que se aplique penalización será el miércoles 19 de marzo a las 23:55. Se debe entregar:

- Memoria respondiendo a todas las preguntas hechas en el guión de prácticas. Este documento pdf incluye los formularios necesarios para responder a todas la preguntas, por lo que se recomienda utilizar el propio documento¹. Aun así, está permitido crear un nuevo formato de memoria, aunque no se recomienda.
- Los códigos en ensamblador implementados y no suministrados con la práctica. En concreto, los códigos implementados en los ejercicios 3.2, 3.4, 4.4, 4.5 y 4.6.

¹Los formularios del PDF pueden no visualizarse con Adobe Acrobat Reader. Si eres usuario de Windows, mi recomendación es utilizar *Okular*, un visor de PDFs gratuito, de software libre y con más funcionalidades que la versión gratuita de Acrobat Reader. Puedes descargar *Okular* aquí: <https://okular.kde.org/es/download/>.

3. Pipeline MIPS segmentado

Caso de estudio A

A continuación, se muestra el código en ensamblador MIPS que realiza la siguiente operación vectorial: $\vec{Z} = \vec{X} + \vec{Y} + a$. Este programa lo puedes encontrar en el fichero `sumaXY.asm`:

```
.data
X: .word 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
   23 24 25 26 27 28 29 30 31 32 33 34 35 36 37
Y: .word 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
   23 24 25 26 27 28 29 30 31 32 33 34 35 36 37
Z: .space 152

.text
la $t0, X
la $t1, Y
la $t2, Z
li $t6, 5100    #escalar a
addi $t3, $t2, 152 #limite del bucle (38 elementos enteros)
loop:
    lw $t4, 0($t0)           #cargar X[i] y Y[i]
    lw $t5, 0($t1)
    add $t5, $t5, $t4
    add $t5, $t5, $t6
    sw $t5, 0($t2)
    addi $t0, $t0, 4          #incrementar punteros
    addi $t1, $t1, 4
    addi $t2, $t2, 4
    bne $t2, $t3, loop
li $v0, 10
syscall
```

Ejercicio 3.1

Carga el programa en el modo IDE, ejecútalo y comprueba la corrección del resultado obtenido. Después ejecuta el programa en el modo pipeline segmentado, configurando el simulador con anticipación completa, resolución de saltos en la etapa ID y predictor de saltos *predict not-taken*. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas y el CPI. ¿A qué se deben los ciclos perdidos que degradan el rendimiento? Razona tu respuesta.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Explicaciones:

Ejercicio 3.2

Planifica el código para que haya el menor número de detenciones utilizando un pipeline con anticipación completa, resolución de saltos en la etapa ID y el tipo de predictor de saltos que prefieras (*predict not-taken*, *predict taken*, o *delayed branch*). Recuerda comprobar la corrección del resultado del programa utilizando el modo IDE. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas, el CPI y el incremento del rendimiento. Razona las modificaciones realizadas y la elección del predictor.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Incremento del rendimiento (%):

Explicaciones:

Caso de estudio B

A continuación, se muestra el código *C* de un algoritmo de ordenación de un vector utilizando el método de selección. En el fichero `ordena.asm` puedes encontrar el código en ensamblador MIPS del algoritmo.

```
for(int i = 0; i < n-1; i++)
    for(int j = i+1; j < n-2; j++)
        if(vec[j] < vec[i]) {
            int tmp = vec[j];
            vec[j] = vec[i];
            vec[i] = tmp;
        }
```

Ejercicio 3.3

Carga el programa en el modo IDE, ejecútalo y comprueba la corrección del resultado obtenido. Después ejecuta el programa en el modo pipeline segmentado, configurando el simulador con anticipación completa, resolución de saltos en la etapa ID y predictor de saltos *predict taken*. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas y el CPI. ¿A qué se deben los ciclos perdidos que degradan el rendimiento? Razona tu respuesta.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Explicaciones:

Ejercicio 3.4

Planifica el código para que haya el menor número de detenciones, utilizando el pipeline con anticipación completa, resolución de saltos en la etapa ID y predictor de saltos *predict taken*. Recuerda comprobar la corrección del resultado del programa utilizando el modo IDE. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas, el CPI y el incremento del rendimiento. ¿Has podido eliminar todas las detenciones? ¿Por qué? Razona tu respuesta.

Ciclos ejecutados:**Nº de Instrucciones:****CPI:****Incremento del rendimiento (%):****Explicaciones:**

Por último, ¿cuál crees que es el predictor más adecuado para este algoritmo? ¿*Predict taken* o *Predict not-taken*? Razona tu respuesta (no es necesario realizar experimentos con diferentes entradas, basta con observar el código fuente MIPS).

Respuesta:

4. Planificación estática de instrucciones

Caso de estudio A

A continuación, se muestra el código en ensamblador MIPS que realiza la multiplicación de un vector por un escalar: $\vec{Y} = \vec{X} * k$. Este programa lo puedes encontrar en el fichero `mult.asm`:

```
.data
X: .word 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
    23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43
    44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63
Y: .space 256
k: .word 10
.text

la $t0, X
la $t7, Y
lw $t1, k
addi $t2, $t7, 256

loop:
        lw    $t3, 0($t0)
        mul   $t3, $t3, $t1
        sw    $t3, 0($t7)
        addi  $t0, $t0, 4
        addi  $t7, $t7, 4
        bne   $t7, $t2, loop

li $v0, 10
syscall
```

Ejercicio 4.1

Carga el programa en el modo IDE, ejecútalo y comprueba la corrección del resultado obtenido. Después ejecuta el programa en el modo pipeline segmentado multiciclo, configurando el simulador con anticipación completa, resolución de saltos en la etapa ID y predictor de saltos *predict taken*. Utiliza la configuración de las unidades multiciclo por defecto (multiplicador segmentado de 3 ciclos y divisor convencional de 6 ciclos). De aquí en adelante, salvo que se indique lo contrario, esta configuración será utilizada en el resto de ejercicios. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas y el CPI. ¿A qué se deben los ciclos perdidos que degradan el rendimiento? Razona tu respuesta.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Explicaciones:

Ejercicio 4.2

Aplica la técnica de *desenrollamiento de bucles* al programa original, reorganizando también el código para reducir todos los tipos de riesgos. Recuerda comprobar la corrección del resultado del programa utilizando el modo IDE. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas, el CPI y el incremento del rendimiento.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Incremento del rendimiento (%):

Ejercicio 4.3

Aplica la técnica de *software pipelining* al programa original. Planifica el resto de instrucciones del programa para minimizar las detenciones. Recuerda comprobar la corrección del resultado del programa utilizando el modo IDE. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas, el CPI y el incremento del rendimiento. ¿Por qué, a pesar de haber eliminado las dependencias de datos, sigue habiendo detenciones en el pipeline? ¿Si operásemos con un vector de números de coma flotante en lugar de enteros, que ocurriría? Razona tu respuesta.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Incremento del rendimiento (%):

Explicaciones:

¿Que pasa si configuramos la latencia del multiplicador a 6 ciclos? ¿El tiempo de ejecución aumenta o disminuye? ¿Por qué? Razona tu respuesta.

Respuesta:

Caso de estudio B

Ejercicio 4.4

Considera una operación vectorial que dados tres vectores $\vec{X}$, $\vec{Y}$ y $\vec{Z}$, de tamaño n , y una constante k , realiza la siguiente operación:

$$Z_i = 2X_i + k \times Y_i, \forall i \in [0, n)$$

Realiza una implementación de esta operación con vectores de 64 enteros sin utilizar ninguna técnica de planificación estática. Recuerda comprobar la corrección del resultado del programa utilizando el modo IDE. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas y el CPI. ¿Hay ciclos perdidos en tu implementación? Razona tu respuesta. Nota: la multiplicación por 2 puede realizarse con *sll* en lugar de con *mul*.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Explicaciones:

Ejercicio 4.5

Aplica la técnica de *desenrollamiento de bucles* al programa anterior, reorganizando también el código para reducir todos los tipos de riesgos. Recuerda comprobar la corrección del resultado del programa utilizando el modo IDE. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas, el CPI y el incremento del rendimiento. ¿Hay ciclos perdidos en tu implementación? Razona tu respuesta.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Incremento del rendimiento (%):

Explicaciones:

Ejercicio 4.6

Aplica la técnica de *software pipelining* al programa del ejercicio 4.4. Planifica el resto de instrucciones del programa para minimizar las detenciones. Recuerda comprobar la corrección del resultado del programa utilizando el modo IDE. Apunta el tiempo de ejecución en ciclos, el número de instrucciones ejecutadas, el CPI y el incremento del rendimiento. ¿Hay ciclos perdidos en tu implementación? Razona tu respuesta.

Nota: Además de aplicar el *software pipelining*, también puedes reordenar el resto de instrucciones del bucle (los incrementos de los punteros) para reducir los riesgos de datos.

Ciclos ejecutados:

Nº de Instrucciones:

CPI:

Incremento del rendimiento (%):

Explicaciones:

Caso de estudio C

Considera el código planificado en el Ejercicio 3.2 (suma de dos vectores). ¿Merecería la pena desenrollar el bucle? ¿Y aplicar la técnica de *software pipelining*? Razona tu respuesta (no es necesario implementar el código, basta con justificar correctamente la respuesta).

Respuesta: